Introducción a Ciencia de Datos

Introducción a Pandas, Numpy y Matplotlib

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Introducción a Pandas

Pandas

- **Pandas** se utiliza para el manejo y análisis de datos
- Facilita trabajar con datos estructurados como **tablas**
- La estructura principal de Pandas es el **DataFrame**
- Soporta formatos comunes como CSV, Excel, JSON, y bases de datos SQL.
- Instalación: `pip install pandas`
- Documentacion: https://pandas.pydata.org/docs/index.html

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Estructuras básicas de Pandas

- **Series**: Son estructuras **unidimensionales**, pueden almacenar datos de cualquier tipo de datos
- La podemos crear a partir de cualquier secuencia (como Listas) de la siguiente manera:

```
[in]    import pandas as pd
        series = pd.Series([10, 20, 30, 40])
        print(series)
```

```
[in]    0    10
        1    20
        2    30
        3    40
        dtype: int64
```

Indice por defecto

Valores

Tipo de datos

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

- **DataFrame**: Son estructuras **bidimensionales** con filas y columnas. Cada columna puede ser de un tipo diferente

- Podemos definir dataframes con la función **DataFrame()**, en este caso vamos a generar uno a partir de un diccionario con listas del mismo largo dentro. Las listas funcionarán como **columnas** y las llaves como los **nombres de las columnas**

```
[in]    data = {
        'Nombre': ['Juan', 'Ana', 'Luis'],
        'Edad': [28, 22, 35],
        'Ciudad': ['Buenos Aires', 'Rosario', 'Córdoba']
        }
        df = pd.DataFrame(data)
        print(df)
```

```
[out]    Nombre  Edad        Ciudad
      0    Juan    28  Buenos Aires
      1     Ana    22       Rosario
      2    Luis    35       Córdoba
```

Nombres de las columnas

Indice por defecto

Valores

✓ Otras formas de definir un DataFrame:

Con una lista de listas y luego definiendo el nombre de las columnas:

```
[in]   data = [
       ['Juan', 28, 'Buenos Aires'],
       ['Ana', 22, 'Rosario'],
       ['Luis', 35, 'Córdoba']
       ]
       df = pd.DataFrame(data, columns=['Nombre', 'Edad', 'Ciudad'])
```

Con una lista de diccionarios, donde cada uno es una fila del DataFrame :

```
[in]   data = [
         {'Nombre': 'Juan', 'Edad': 28, 'Ciudad': 'Buenos Aires'},
         {'Nombre': 'Ana', 'Edad': 22, 'Ciudad': 'Rosario'},
         {'Nombre': 'Luis', 'Edad': 35, 'Ciudad': 'Córdoba'}
         ]
         df = pd.DataFrame(data)
```

✓ Otra forma de visualizarlo en notebooks es con: `display(df)`

✓ Google Colab ofrece opciones de análisis automáticos desde esta función

	Nombre	Edad	Ciudad
0	Juan	28	Buenos Aires
1	Ana	22	Rosario
2	Luis	35	Córdoba

Operaciones comunes con Pandas

- **Lectura de archivos**:

  ✓ CSV: Utilizando **read_csv()**

```
[in]  df = pd.read_csv('archivo.csv')
```

Podemos añadir parámetros para especificar la codificación o la separación en el documento

```
[in]  df = pd.read_csv('archivo.csv', sep=';', encoding='utf-8')
```

  ✓ Excel: Utilizando **read_excel()**

```
[in]  df = pd.read_excel('archivo.xlsx')
```

Podemos añadir parámetros para especificar la hoja del excel o las columnas que necesitamos

```
[in]  df = pd.read_excel('archivo.xlsx',
                         sheet_name='hoja_1', usecols=['col_1', 'Col_2'])
```

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

- **Crear una conexión con SQLAlchemy**:

```
[in]    from sqlalchemy import create_engine

        # Definir la conexión a la base de datos
        engine = create_engine('postgresql+psycopg2://usuario:contraseña@localhost:5432/nombre_db')
```

- **Ejecutar una consulta SQL**: Con la función **read_sql()**

```
        # Leer datos desde la base de datos usando pandas
        query = "SELECT * FROM tabla"
        df = pd.read_sql(query, con=engine)
```

- **Subir un DataFrame a la Base de Datos**: Con la función **to_sql()**

```
        df.to_sql('nueva_tabla', con=engine, if_exists='replace', index=False)
```

- **Acceder a los datos**:

  ✓ Columna: Una forma es indexarle el nombre de la columna al DataFrame

```
[in]   print( df['Nombre'] )# ó df.Nombre
```

```
[out]  0    Juan
       1     Ana
       2    Luis
       Name: Nombre, dtype: object
```

Noten que lo que devuelve indexar una columna es una **Serie**, cuyo nombre está definido como el nombre de la columna

```
print( df[ ['Nombre' , 'Edad'] ] )
```

También podemos indexar varias columnas al mismo tiempo

  ✓ Fila (por **índice**): Utilizando **`.iloc[`***`indice_fila`***`]`**

```
[in]   print(df.iloc[0])
```

```
[out]  Nombre            Juan
       Edad                28
       Ciudad    Buenos Aires
       Name: 0, dtype: object
```

Noten que el nombre de la Serie que devuelve indexar la primera fila es el nombre del indice, que en este caso al ser predeterminado es el 0

- **Filtrar datos**: Podemos indexarle a un DataFrame una Serie de Booleanos que filtra qué columnas se van a quedar. Veamos algunos Ejemplos:

```
[in]    df_filtrado = df[ df['Edad'] > 30
]
```

```
[in]    df_filtrado = df[df['Ciudad'].isin(['Córdoba', 'Mendoza'])]
```

- **Eliminar Datos Nulos:** Podemos especificar la columna añadiendo el argumento `subset`, sino eliminará toda fila con un valor nulo en cualquier columna

```
[in]    df_sin_na = df.dropna(subset=['Ciudad'])
```

A la mayoría de los métodos que cambian el estado de un DataFrame podemos incluirle el argumento `inplace = True` que realiza el cambio en la variable original sin tener que declarar una variable nueva

```
[in]    df.dropna(subset=['Ciudad'], inplace = True)
```

- **Agregar nuevas filas**:

  ✓ Con **loc[ ]**: Entre los corchetes elegimos el indice de la nueva columna. Si ya existe una fila con ese índice, será sobrescrita, sino será agregada.

```
[in] # Añadimos una fila al final del DataFrame
     df.loc[len(df)] = ['María', 30, 'Mendoza']
```

Noten que añadimos una lista en donde no especificamos a que columna pertenece cada valor por lo que se asignan siguiendo el orden de las columnas.

  ✓ Con **concat( )**: Esta función concatena dos DataFrames, podemos armar uno de una sola fila utilizando un diccionario.

```
[in]   # Crear un nuevo DataFrame con la fila a agregar
       nueva_fila = pd.DataFrame({'Nombre': 'María', 'Edad': 30, 'Ciudad': 'Mendoza'})

       # Concatenar el nuevo DataFrame al original
       df = pd.concat([df, nueva_fila], ignore_index=True)
```

Usamos `ignore_index=True` para reindexar el DataFrame de manera consecutiva.

El orden de las columnas en la nueva fila no importa siempre que los nombres de las columnas coincidan con los del DataFrame original

- **Agregar nuevas columnas**: Podemos hacerlo indexando el DataFrame con el nombre de la nueva columna y asignándole un valor. Veamos distintas formas de hacerlo.

Valor constante:

```
[in]  # Esto agrega una columna con un valor fijo en sus filas
      df['Nacionalidad'] = 'Argentina'
```

Con operaciones (podemos hacer operaciones entre distintas filas):

```
[in]   # Podemos agregar una columna alterada por una operación
       df['Edad en 10 años'] = df['Edad'] + 10
```

Aplicando una función:

```
[in]  # Función que clasifica según la edad
      def clasificar_edad(edad):
          if edad < 30:
              return 'Joven'
          else:
              return 'Adulto'

      # Agregamos una nueva columna basada en la función
      df['Clasificación'] = df['Edad'].apply(clasificar_edad)
```

Transformación de datos

apply(): permite aplicar una función a lo largo de un eje del DataFrame (**filas o columnas**)

```
import pandas as pd

# Crear un DataFrame
df = pd.DataFrame({
    'A': [1, 2, 3, 4],
    'B': [10, 20, 30, 40]
})

# Aplicar una función a cada columna
df['A'] = df['A'].apply(lambda x: x * 2)
```

	A	B
0	2	10
1	4	20
2	6	30
3	8	40

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Transformación de datos

applymap(): se utiliza para aplicar una función **elemento a elemento** en todo el DataFrame.

```
import pandas as pd

# Crear un DataFrame
df = pd.DataFrame({
    'A': [1, 2, 3, 4],
    'B': [10, 20, 30, 40]
})

# Aplicar una función a cada columna
df.applymap(lambda x: x * 2)
```

	A	B
0	2	20
1	4	40
2	6	60
3	8	80

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Transformación de datos

map(): es utilizado en Series para aplicar una función o un diccionario de mapeo a sus valores.

```
# Crear una Serie
s = pd.Series([1, 2, 3, 4])

# Mapeo usando un diccionario
s.map({1: 'A', 2: 'B', 3: 'C', 4: 'D'})
```

```
0    A
1    B
2    C
3    D
```

Tablas Pivot con Pandas

- Las **tablas pivot** son una poderosa herramienta para resumir, reorganizar y visualizar datos. Nos permiten reorganizar filas y columnas para observar la relación entre diferentes variables.

- Reorganiza datos tabulares y permite agregar valores, proporcionando una vista resumida de los datos.

- Para crear una tabla pivot en Pandas, utilizamos el método **pivot_table()**, que nos permite agregar los datos en función de las filas y columnas especificadas.

Tablas Pivot con Pandas

```python
import pandas as pd

# Crear un DataFrame
data = {
    'Año': [2021, 2021, 2022, 2022],
    'Producto': ['A', 'B', 'A', 'B'],
    'Ventas': [100, 150, 200, 250]
}
df = pd.DataFrame(data)

# Crear una tabla pivot
tabla_pivot = pd.pivot_table(df,
values='Ventas', index='Año',
columns='Producto', aggfunc='sum')
print(tabla_pivot)
```

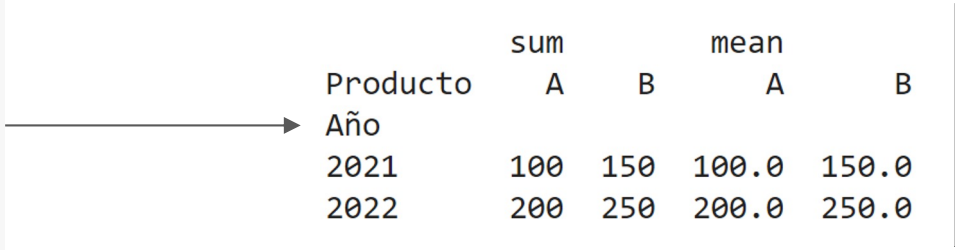

Producto	A	B
Año		
2021	100	150
2022	200	250

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Tablas Pivot con Pandas

- Las tablas pivot permiten usar varias funciones de agregación como **sum(), mean(), count()**, etc. para sumarizar los datos.

```
# Tabla pivot con múltiples funciones de agregación
tabla_pivot = pd.pivot_table(df, values='Ventas', index='Año', columns='Producto', aggfunc=['sum', 'mean'])
print(tabla_pivot)
```

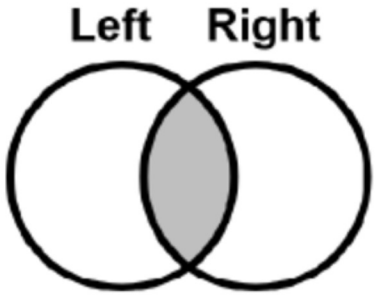


	sum		mean	
Producto	A	B	A	B
Año				
2021	100	150	100.0	150.0
2022	200	250	200.0	250.0

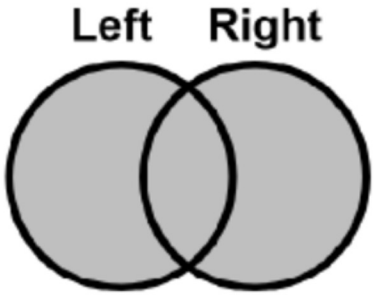
- **aggfunc:** Define la función que se aplicará a los valores agregados (suma, promedio, conteo, etc.).

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

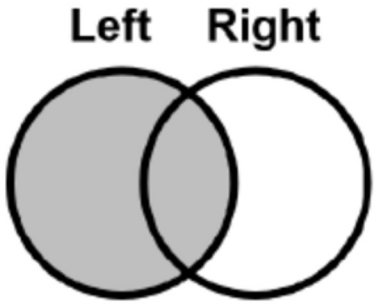
- **Unión de columnas (SQL join)**: Podemos usar la función **merge()** para agregar una columna a un DataFrame procedente de otro, emulando un **join** de tablas al estilo SQL. Es importante que ambos DataFrames **compartan una columna** que servirá de **pivote**. Veamos los tipos de join que podemos hacer:



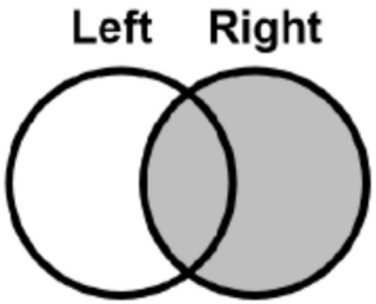
INNER JOIN



FULL OUTER JOIN



LEFT OUTER JOIN



RIGHT OUTER JOIN

Veamos un ejemplo de **left join**:

```
[in]    df_puntos = pd.DataFrame({'Nombre' : ['Juan', 'Nicolas', 'Ana', 'Facundo', 'Luis'],
                                  'Puntos' : [12, 34, 19, 45, 23]})

        df_left = pd.merge(df, df_puntos, on='Nombre', how='left')

        print(df_left)
```

El primer argumento es el DataFrame a la izquierda

Luego el de la derecha

En **on** asignamos el o los pivotes

Por ultimo en **how** especificamos el tipo de join

```
[out]   Nombre  Edad        Ciudad  Edad en 10 años  Puntos
      0   Juan    28  Buenos Aires             38.0    12.0
      1    Ana    22       Rosario             32.0    19.0
      2   Luis    35       Córdoba             45.0    23.0
      3  María    30       Mendoza              NaN     NaN
```

Noten que en las filas donde no hay valor asignado, Pandas automáticamente reemplaza el valor con un `NaN`, por lo que es un valor **Nulo**. También pasa cuando agregamos una fila y no especificamos qué valor tiene en cierta columna, como en '`Edad en 10 años`'

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Exploración de datos

Tipos de atributos/Features

- **Features Categóricas/Discretas:**
  Cualitativos, Discretos, Operaciones limitadas.
  - **Nominales y Binarios:** Sin orden. Igualdad y desigualdad.
  - **Ordinales:** Orden definido. Igualdad, desigualdad, mayor y menor.
- **Features Numéricas:**
  Cuantitativos, Continuos o Discretos, Números y tratados como tales. Algunos autores hacen la distinción entre features acotadas e infinitas.
  - Operaciones anteriores, sumas, restas. (a veces multiplicación/ división)

Existen atributos que pueden parecer numéricos pero son categóricos.

Ejemplos de tipos de atributos

- Cantidad de televisores en la casa
- Tamaño de un combo (Pequeñas, Medianas, Grandes)
- Número de llamadas realizadas durante un mes
- Duración de la llamada más larga
- Precio de un libro
- Código postal
- Temperatura en grados Centígrados
- Temperatura en grados Farenheit
- Temperatura en grados Kelvin

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Ejemplos de tipos de atributos

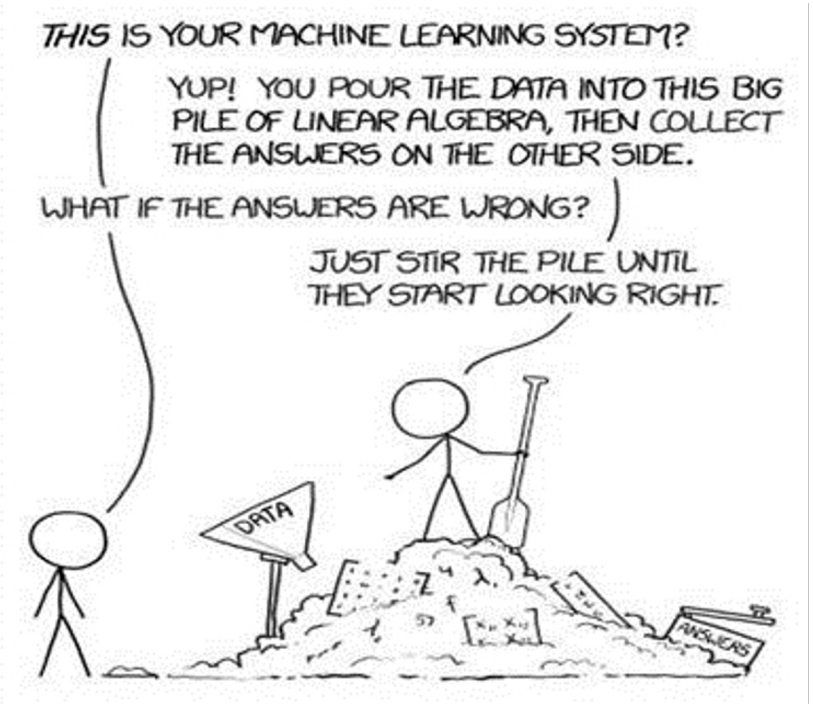
- Cantidad de televisores en la casa: Numérico, discreto.
- Tamaño de un combo (Pequeñas, Medianas, Grandes): Categórico (ordinal), discreto.
- Número de llamadas realizadas durante un mes: Numérico, discreto.
- Duración de la llamada más larga: Numérico, continuo.
- Precio de un libro: Numérico, discreto (trabajemos con dos decimales).
- Código postal: Categórico (nominal), discreto.
- Temperatura en grados Centígrados: Numérico, continuo.
- Temperatura en grados Farenheit: Numérico, continuo.
- Temperatura en grados Kelvin: Numérico, continuo.

Operaciones por tipos de atributos

Operación	Nominal	Ordinal	Numérico
Ejemplos	*Sexo ('F' o 'M')*	*Calificación (E, MB, B, R)*	*Temp. en Celsius (23 °C…)*
Distribution & Frecuencia	Si	Si	Si
Mediana, percentiles	No	Si	Si
Suma, resta	No	No	Si
Media/promedio, Desviación estándar	No	No	Si
Ratios, Porcentajes de variación	No	No	Si

Limpieza & Preparación de Datos

4. Limpieza y preparación de datos



*No es lo más divertido, pero puede llegar a ser **MUY** necesario*

Valores Faltantes

- Interfiere en los comportamientos y relaciones a aprender por el algoritmo.
- Muchos algoritmos no pueden trabajar con features (valores) faltantes

Name	Weight	Gender	Play Cricket/ Not
Mr. Amit	58	M	Y
Mr. Anil	61	M	Y
Miss Swati	58	F	N
Miss Richa	55		Y
Mr. Steve	55	M	N
Miss Reena	64	F	Y
Miss Rashmi	57		Y
Mr. Kunal	57	M	N

Gender	#Students	#Play Cricket	%Play Cricket
F	2	1	50%
M	4	2	50%
Missing	2	2	100%

Name	Weight	Gender	Play Cricket/ Not
Mr. Amit	58	M	Y
Mr. Anil	61	M	Y
Miss Swati	58	F	N
Miss Richa	55	F	Y
Mr. Steve	55	M	N
Miss Reena	64	F	Y
Miss Rashmi	57	F	Y
Mr. Kunal	57	M	N

Gender	#Students	#Play Cricket	%Play Cricket
F	4	3	75%
M	4	2	50%

Valores Faltantes

Estrategias:

- **Descartar mediciones**
  Remover del dataset mediciones completas a las que le falte una feature.
- **Descartar atributo**
  Remover el atributo completo de todas la mediciones, removiendolo del análisis.
- **Definir un valor**
  Completar los feature faltantes con un valor por default: cero, promedio, mediana, moda, etc.
- **Predecir un valor**
  Completar mediante el uso de un modelo entrenado con el subconjunto de datos completos.

Name	Weight	Gender	Play Cricket/ Not
Mr. Amit	58	M	Y
Mr. Anil	61	M	Y
Miss Swati	58	F	N
Miss Richa	55		Y
Mr. Steve	55	M	N
Miss Reena	64	F	Y
Miss Rashmi	57		Y
Mr. Kunal	57	M	N

Valores Faltantes

1. Remover mediciones con valores nulos en un atributo:
   ```
   unDataFrame.dropna(subset=[“atributo”])
   ```
2. Remover un atributo (columna) del DataFrame:
   ```
   unDataFrame.drop(columns=[“atributo”])
   ```
3. Definir un valor, por ejemplo la media:
   ```
   mediaDelAtr = unDataFrame[“atributo”].median()
   unDataFrame[“atributo”].fillna(mediaDelAtr)
   ```
4. Remover duplicados:
   ```
   unDataFrame.drop_duplicates(inplace=True)
   ```

Atributos Categóricos

- Algoritmos suelen **preferir atributos numéricos** (Por los cálculos que hacen)

Cómo los convertimos?

ID	Color
1	Azul
2	Negro
3	Azul
4	Blanco
5	Blanco

Label Encoding:

ID	Color
1	0
2	1
3	0
4	2
5	2

One-hot-encoding:

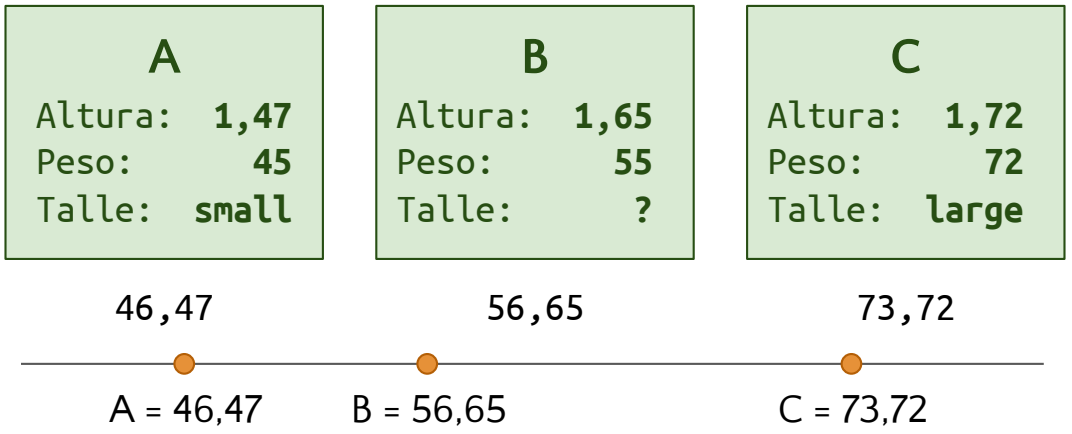
ID	Azul	Negro	Blanco
1	1	0	0
2	0	1	0
3	1	0	0
4	0	0	1
5	0	0	1

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Feature Scaling

- Atributos en **diferentes escalas afectan negativamente** la performance.

Nombre Atributo	Altura (m.)	Peso (kg.)	Sueldo (ARS)
Min.	1,47	45	8.300,0
Max.	1,72	72	105.000,0

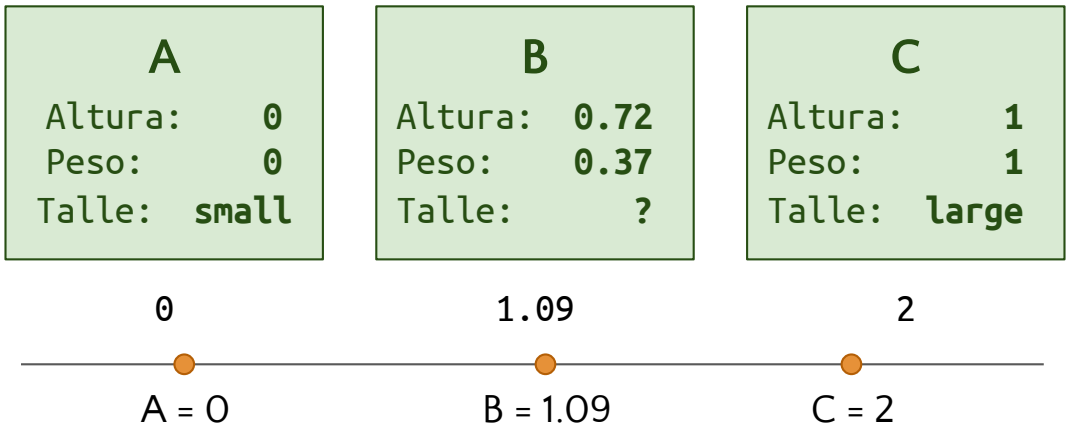


UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Feature Scaling

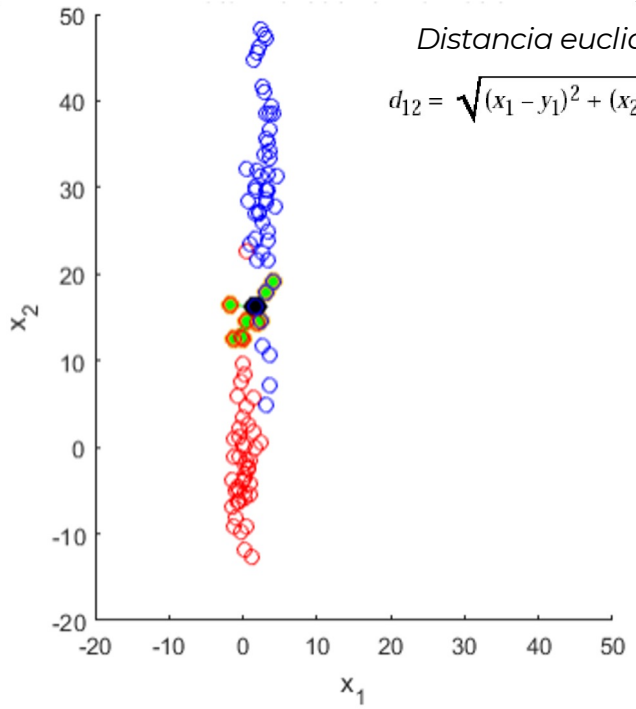
- Atributos en **diferentes escalas afectan negativamente** la performance.

Nombre Atributo	Altura (m.)	Peso (kg.)	Sueldo (ARS)
Min.	0	0	0
Max.	1	1	1



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Feature Scaling

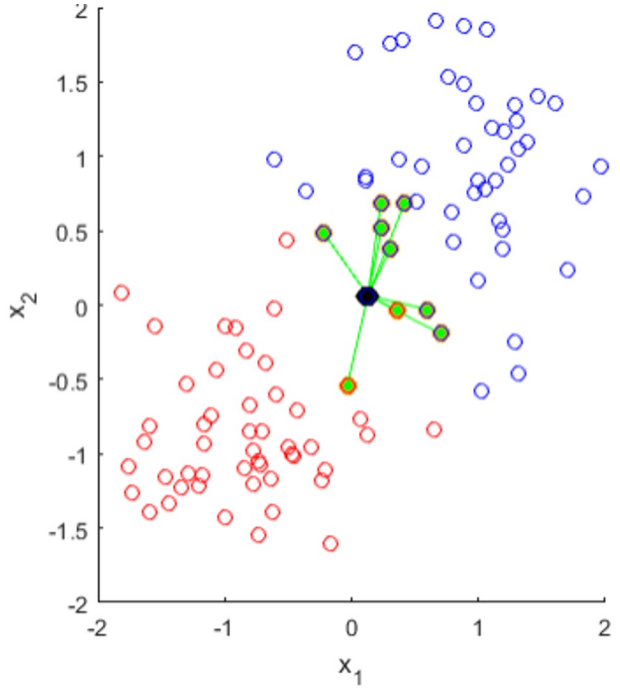


Distancia euclidiana

$$d_{12} = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

Sin escalado

Pero si escalamos...



Escalado

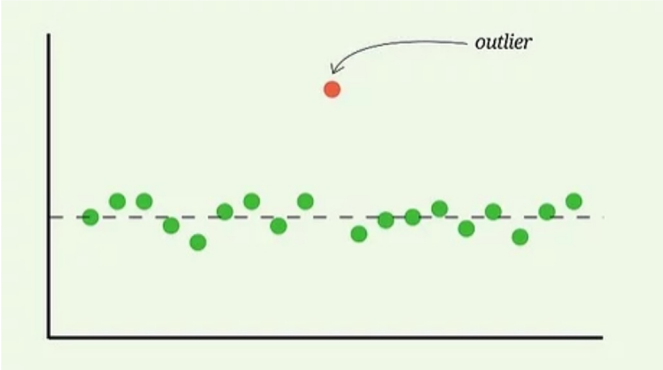
Feature Scaling

Estrategias:

- **Normalización/Min-Max:**
  [min, max] → [0, 1]
  Rango Fijo.
  - Normalización min-max (entre 0 y 1)
  - Es útil aunque los datos no sigan una distribución normal, pero es muy sensible a outliers (porque aparecen max y min)

$$x_{new} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

donde:
x: valor a normalizar
x_{new}: valor normalizado
x_{min}: mínimo del att.
x_{max}: máximo del att.



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Feature Scaling

Estrategias:

- **Estandarización:**
  - Rango variable, problemas en algunos algoritmos.
  - **Menos susceptible a outliers**/valores extremos.
  - No es demasiado interpretable para datos con distribuciones que no sean gaussianas

$$x_{new} = \frac{x - \mu}{\sigma}$$

donde:
x: valor a normalizar
x_{new}: valor normalizado
μ: media del att.
σ: varianza del att.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Preparación de datos

Preparación manual vs funciones.

- Aplicar las transformaciones reiteradas veces.
- Reutilizar funciones en otros dataSet y/o proyectos.
- Migrar y utilizarlas en Producción.
- Probar varias transformaciones de forma simple.

Visualización de datos

Visualización de datos

- A la hora de explorar los datos con los que nos enfrentamos o cuando queremos comunicar nuestros análisis a determinada audiencia, la **visualización** aparece como una de las herramientas fundamentales

- Entonces, es necesario entender el **contexto** para el cual necesitamos la visualización, si la misma va ir acompañada de información extra. Además, decidir la correcta herramienta de visualización y la **estética** necesaria para resaltar lo que queremos indicar.

Librería de visualización



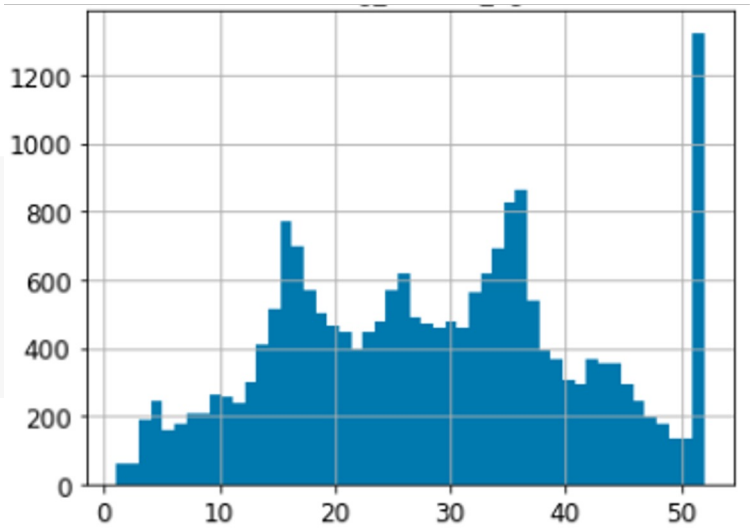
- **Generar gráficos** a partir de código python:
  - Gráficos de barra, histogramas, scatterplots, error-charts, etc...
- Puede ser usados en distintos entornos como:
  - Scripts de python
  - Web servers
  - *Jupiter notebooks*

- Open source, licencia PSF.
- https://matplotlib.org/

```
[in]    import matplotlib.pyplot as plt
```

Librería de visualización

```
%matplotlib inline
# only in a Jupyter notebook
import matplotlib.pyplot as plt
housing.hist(bins=50, figsize=(20,15))
plt.show()
```



Lo vemos mejor en la práctica...

Librería de visualización

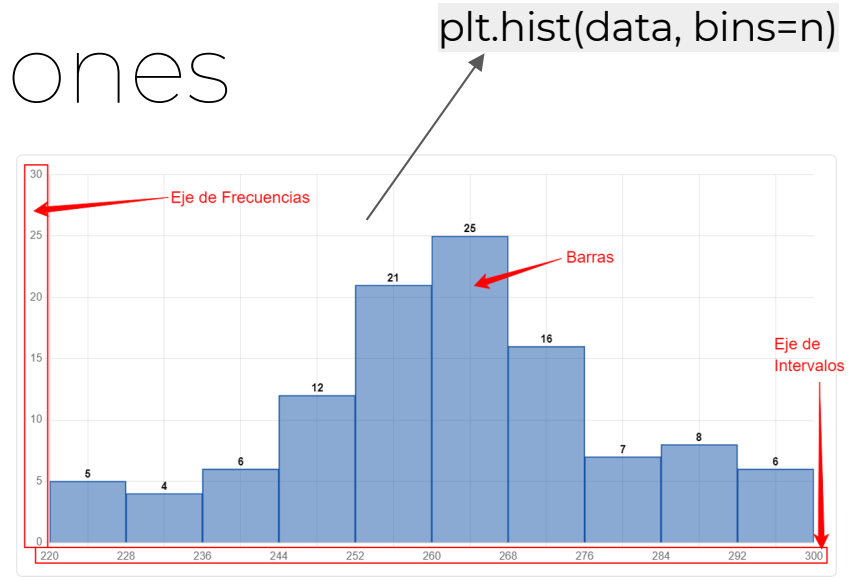


- Estilo gráfico mejorado por defecto en comparación con Matplotlib.
- Integración sencilla con Pandas para el manejo de datos.
- Capacidad para crear gráficos complejos de manera más intuitiva.
- https://seaborn.pydata.org/

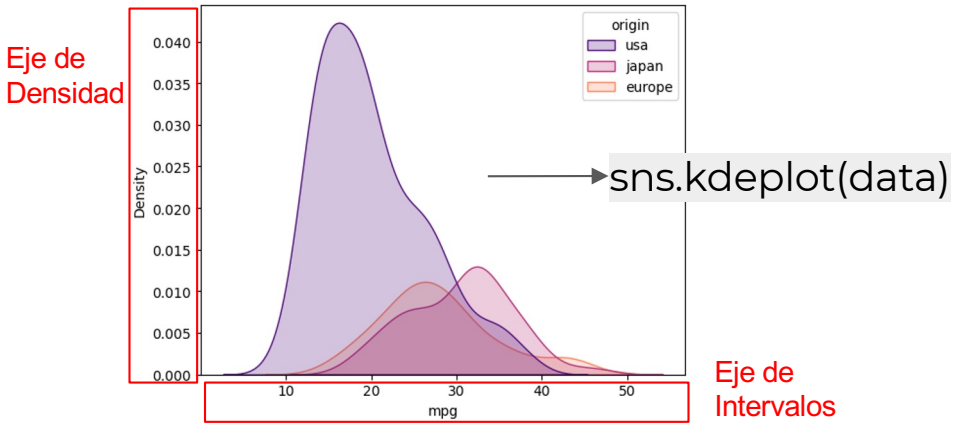
```
[in]    import seaborn as sns
```

Ejemplos - Distribuciones

Histograma: Muestra la distribución de una variable numérica dividiendo los datos en **intervalos** (bins) y contando cuántos valores caen en cada intervalo. Es útil para entender la **frecuencia** de los datos y la forma general de su distribución



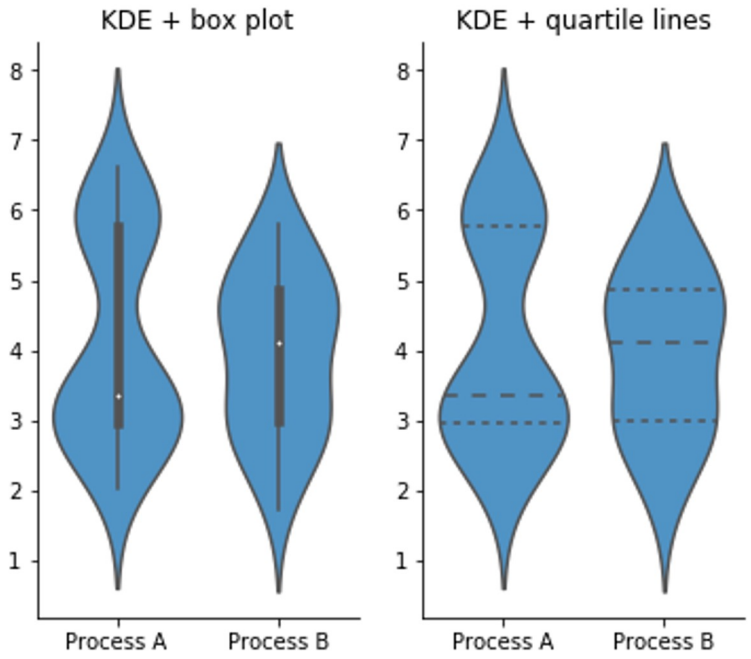
KDE plot (Kernel Density Estimation):
Es una estimación suave de la **densidad** de probabilidad de una variable. Es útil para visualizar la distribución subyacente de los datos sin depender del tamaño de los bins. Logrando una **vista** más **detallada** de la **distribución**



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Ejemplos - Distribuciones

Violin Plot: Combina características de un **boxplot** y un **KDE** plot para mostrar la distribución de una variable numérica.



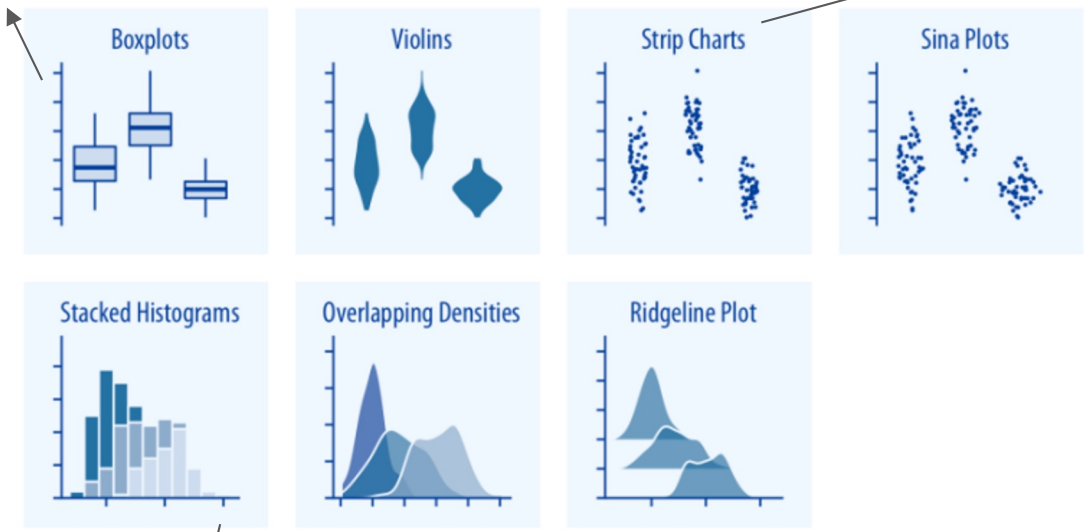

```
sns.violinplot(x='categoría', y='valor', data=data)
```

Ejemplos - Distribuciones

Tenemos distribuciones de ciertos campos, y podemos llegar a tenerlos diferenciados según categorías

`sns.boxplot(x='categoría', y='valor', data=data)`

`sns.stripplot(x='categoría', y='valor', data=data)`



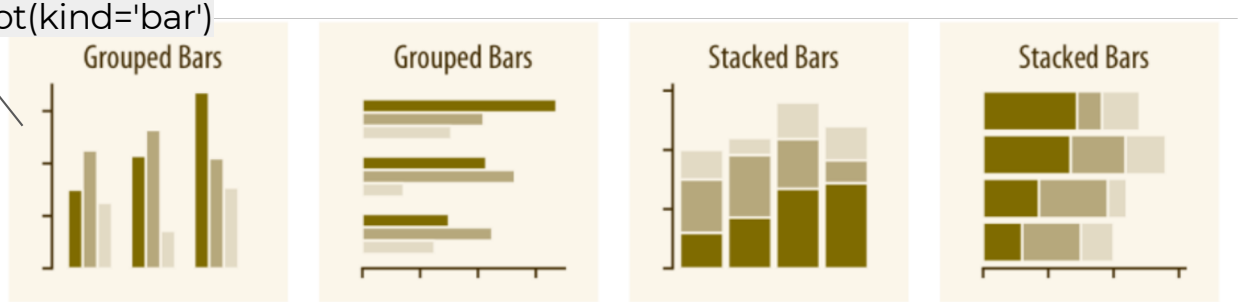

```
data.pivot_table(index='categoría', columns='subcategoría',
values='valor', aggfunc='sum').plot(kind='bar', stacked=True)
```

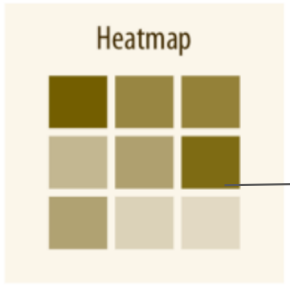
UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Ejemplos - Cantidades y Categorías

Tenemos una cantidad, cuyo valor depende de distintas categorías. En general, alguna de los datos categóricos es ordenable (ej. tamaños de una prenda)

```
data.pivot_table(index='categoría', columns='subcategoría', values='valor', aggfunc='sum').plot(kind='bar')
```



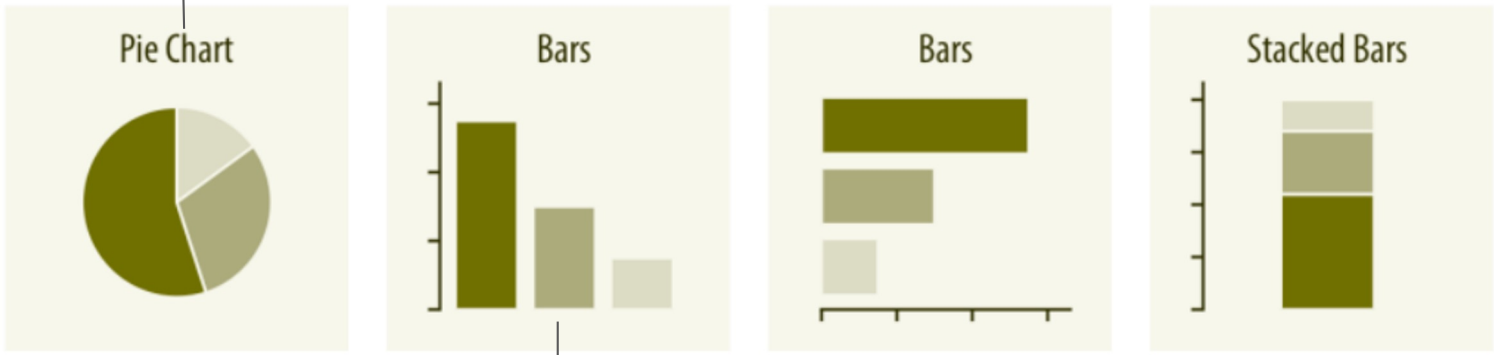



```
sns.heatmap(data.corr(), annot=True, cmap='coolwarm')
```

Ejemplos - Proporciones

Similar al caso de cantidades/categorías, pero en los valores no dependen de la categoría

```
plt.pie(data['valor'], labels=data['categoría'], autopct='%1.1f%%')
```




```
plt.bar(data['categoría'], data['valor'])
```

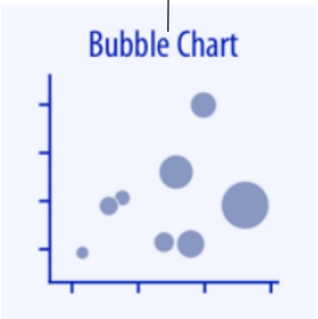
Ejemplos - X vs Y

Tenemos datos numéricos y queremos graficar cómo se relacionan. Si los pares (x,y) pueden diferenciarse por categorías, podemos marcarlo con colores o formas

`plt.scatter(x,y)`



`plt.scatter(data['variable_x'], data['variable_y'], s=data['tamaño'], alpha=0.5)`







UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES